

TRABAJO PRÁCTICO

REDES NEURONALES

- **Asignatura:** Matemática III
- **Período Lectivo:** 1er Cuatrimestre 2026
- **Cátedra:** B
- **Integrantes:** Napoli Bruno
- **Carrera:** Tecnicatura en Programación Informática
- **Fecha de Entrega:** Viernes 05 de Junio de 2026
- **Facultad:** UNSAM
- **Objetivo:** El presente trabajo práctico tiene como objetivo el diseño, análisis e implementación de un modelo de Red Neuronal para la clasificación binaria de transacciones comerciales. Utilizando un conjunto de datos de ventas de comercio electrónico (*Online Retail*), se busca predecir si una transacción corresponde a un perfil de cliente mayorista o minorista

Parte 1

- a) - “**InvoiceNo**”: es el número de factura de cada operación. A veces puede aparecer una “C” que indica una cancelación. Su tipo de dato: Categórica Nominal

Justificación: es un identificador de grupo, no se realiza ninguna operación con dicho dato.

-“**StockCode**”: es el número único de identificación de cada producto. Su tipo de dato: Categórica Nominal

Justificación: identificador del producto, no se realiza ninguna operación.

-“**Description**”: es el nombre/descripción del producto. Su tipo de dato: Categórica Nominal (texto libre).

Justificación: es texto que sirve para describir el producto, no será utilizado en la red.

-“**Quantity**”: es el número que representa la cantidad comprada del producto. Su tipo de dato: Numérica Discreta.

Justificación: es un número entero (no se vende de a fracciones, algo ilógico). En caso de estar negativo representa una devolución. Será un valor que debe ser normalizado.

-“**InvoiceDate**”: es la fecha en la que se realizó la operación. Su tipo de dato: fecha (Datetime).

Justificación: no es número, en la práctica será transformada en algún número para que la red pueda utilizarla pero no como valor de entrada

-“**UnitPrice**”: es el número que representa el valor unitario del producto. Su tipo de dato: Numérica Continua.

Justificación: valor que admite decimales, valores en libras esterlinas. Será un valor que debe ser normalizado.

-“**CustomerID**”: es el valor que representa el código único de cada cliente.

Su tipo de dato: Categórica Nominal.

Justificación: valor que sirve para identificar a cada cliente, no se realiza ninguna operación con ella.

-“**Country**”: nombre del país donde se realiza la operación. Su tipo de dato: Categórica Nominal.

Justificación: es una variable que representa el país donde se realiza dicha operación. Será modificada a valor binario: 1 para United Kingdom y 0 para el resto.

- b) La más influyente sería “**Quantity**”, porque el mayorista compraría en mayor cantidad. **UnitPrice** no es determinante, pero podría serlo porque tal vez algunos ya tienen menor precio unitario al comprar en cantidades. De por sí sola no es. También podría tomar el tipo de producto, porque no es lo mismo comprar muchas cosas de algo pequeño o menor precio a que comprar algo grande en cantidades y/o valor muy elevado. “**Country**” podría depender del país, tal vez algunos tienen mayores mayoristas que otros. El resto de las variables las descarto porque no aportan.

- c) Mi base de datos, para una red neuronal de clasificación binaria, sí es adecuada. Tiene su variable objetivo con 2 clases posibles (mayorista o minorista), predice si se trata de una compra mayorista o minorista. El resultado final binario sería 0 para minorista y 1 para mayorista.

El objetivo sería poder aplicar diferentes precios a las compras mayorista y mejorarles el precio.

- d) Valores atípicos: hay ventas con muchísimas cantidades, por lo que pongo como tope a 1000. Todo lo que supere los 1000 lo descarto porque considero que es un valor improbable en la realidad.

-En los precios unitarios elimino aquellos que tengan valor 0, o nulo, porque no me sirve para el aprendizaje.

-Las facturas canceladas las elimino (poseen una C), a mí me interesan saber todas las que se vendieron.

-Las devoluciones también las elimino por la misma razón de antes.

-Los precios unitarios altos los mantengo porque hay productos muy caros y con eso puedo estudiar si se compran muchos o pocos pese al precio.

*métodos utilizados se encuentran en el archivo de Python donde está todo el trabajo.

- e) Normalizaría las columnas ‘**Quantity**’ y ‘**UnitPrice**’, porque tienen valores muy grandes y podría afectar negativamente el entrenamiento. Para esta parte uso Z-Score por tener valores extremos.

Parte 2

a) La red neuronal cuenta con 3 neuronas de entrada:

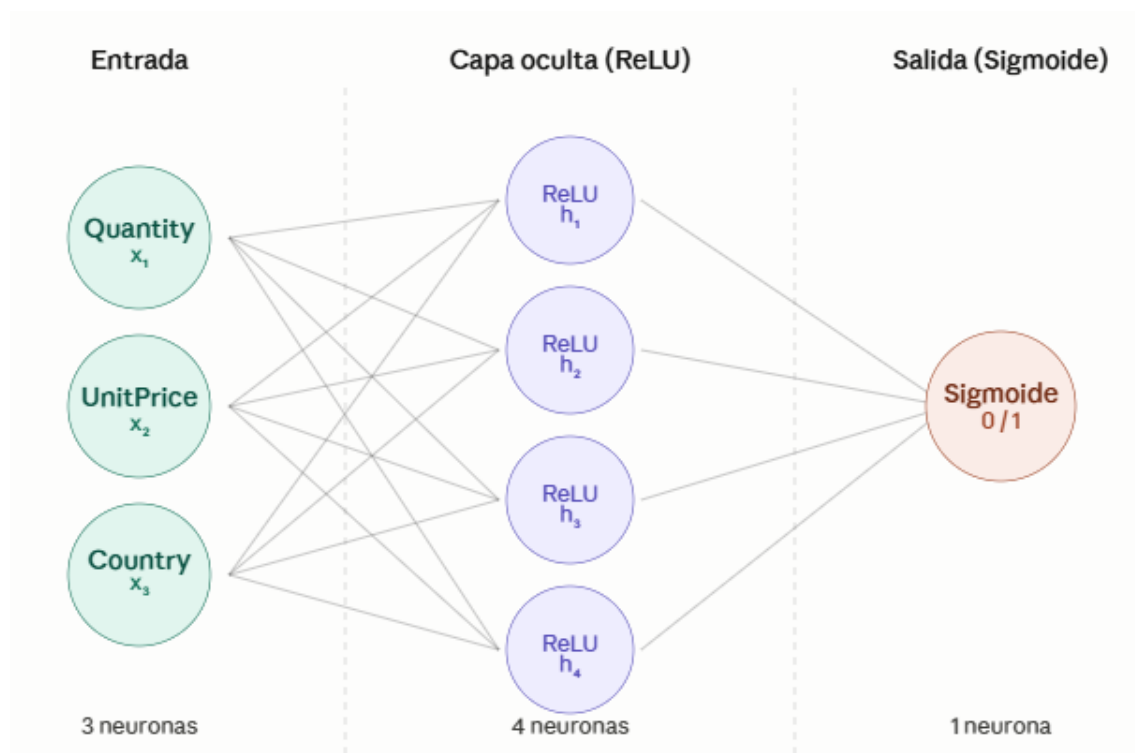
-Quantity

-UnitPrice

-CountryBinario (previamente modificado para su utilización)

La capa oculta sería 1 con 4 neuronas con función de activación *ReLU*

La capa de salida será de 1 neurona y su función de activación será de *Sigmoide* para transformar la salida en un valor de 0 o 1 para mayorista y minorista



b) y c) El código completo se encuentra en el archivo Python.

d) Al correr el código, de 145 mayoristas reales predijo correctamente 142, equivocándose solo en 3. Al analizar los gráficos, ambas curvas (entrenamiento y test) siguen trayectorias similares tanto en la función de pérdida como en el accuracy, por lo que el modelo no presenta overfitting. Esto se confirma al detectar 142 de 145 mayoristas, lo que demuestra que el modelo aprendió genuinamente a distinguir entre ambas clases.

El overfitting se identificaría si la curva de test empezara a subir mientras la de entrenamiento sigue bajando. En ese caso lo podría manejar deteniendo el entrenamiento en ese punto, o reduciendo la cantidad de neuronas de la capa oculta.

Cabe recalcar que las corridas varían por la inicialización aleatoria.

Parte 3

a) El código completo se encuentra en el archivo Python.

b) En ambos códigos, tanto Numpy como Sklearn obtengo resultados similares, un 99% aprox.

-En numpy obtengo un promedio de 99.7% tanto en entrenamiento como en test.

-En Sklearn obtengo un promedio de 99.92% en entrenamiento y en test un 99.88%.

Por momentos, en Numpy, obtengo un mayor resultado que en Sklearn, pero esto depende de la inicialización aleatoria de los pesos y sesgos. En Sklearn tiende a ser más consistente entre corridas porque internamente utiliza otras técnicas de optimización más robusta.

c)

Configuración		Train	Test
4 neuronas, 100.000 iteraciones,	L=0.05	99.92%	99.88%
2 neuronas, 100.000 iteraciones,	L=0.05	99.95%	99.79%
4 neuronas, 20.000iteraciones,	L=0.05	99.95%	99.82%
4 neuronas, 20.000 iteraciones,	L=0.5	99.94%	99.97%
4 neuronas, 20.000 iteraciones,	L=0.001	99.44%	99.42%

Se probaron diferentes configuraciones modificando la cantidad de neuronas, iteraciones y tasa de aprendizaje. La única configuración que mostró una diferencia notable fue con una tasa de aprendizaje muy baja (L=0.001), donde el accuracy bajó a 99.4%, debido a que la red aprende muy lentamente y no llega a aprender completamente todo en 20.000 iteraciones.

Parte 4

Para la primera parte, fue importante sacar de encima todos los productos que no tengan precio, y aquellas transacciones que hayan sido canceladas. De encontrarse esos valores en mi red neuronal, no hubiesen ayudado en nada ya que no me interesaba si habían sido canceladas o valores 0.

Al aplicarla manualmente aprendí el paso a paso de lo que hace una red neuronal, y lo importante de los datos que voy anotando para que pueda aprender correctamente.

Conclusión:

El modelo funcionó muy bien, alcanzando un accuracy cercano al 99%. Sin embargo, como mejora futura se podría incorporar la frecuencia de compra por cliente como criterio adicional para definir mayorista, combinando tanto la cantidad por transacción como la recurrencia del cliente.

Referencias

- Dataset: Online Retail Dataset — UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/352/online+retail>
- Ejemplo base de red neuronal: material de cátedra provisto por los docentes (Unidad 5)